

Fonctions, Paramètres et Gestion des Erreurs

Consignes générales

L'ensemble des étapes doit être implémenté dans un fichier de script PowerShell nommé `audit_securise.ps1`.

Le travail s'effectue dans un éditeur de code (VS Code ou PowerShell ISE).

L'objectif est d'industrialiser le script en le rendant dynamique via des arguments de ligne de commande, en encapsulant la logique d'écriture dans une fonction réutilisable et en interceptant les erreurs d'exécution pour éviter tout arrêt inopiné.

1. Déclaration des paramètres d'exécution

- Déclarer un bloc de paramètres `param()` en en-tête de script.
- Définir un premier paramètre `$DossierCible` de type chaîne de caractères (`[string]`), avec la valeur par défaut `"C:\Windows"`.
- Définir un second paramètre `$SeuilTailleMo` de type nombre entier (`[int]`), avec la valeur par défaut 50.
- Définir un troisième paramètre `$ModeDetaille` sous forme de commutateur logique (`[switch]`).
- Afficher un message d'initialisation en couleur cyan dans la console, reprenant la valeur effective de chaque paramètre.

2. Conception d'une fonction de journalisation

- Déclarer à l'intérieur du script une fonction nommée `Write-LogMessage`.
- Configurer deux paramètres obligatoires pour cette fonction :
 - `$Message` : le texte de l'événement (chaîne de caractères).
 - `$Niveau` : la gravité de l'événement, restreinte strictement aux valeurs `Information`, `Avertissement` et `Erreur` via un attribut de validation.
- Définir la cible de journalisation vers `C:\Audit\audit.log`.
- S'assurer que le répertoire `C:\Audit` est créé automatiquement s'il n'existe pas.
- Préfixer chaque ligne consignée par un horodatage précis au format `yyyy-MM-dd HH:mm:ss`, suivi du niveau de gravité.
- Écrire chaque enregistrement à la fin du fichier sans écraser les données précédentes, avec un encodage UTF-8.
- Afficher simultanément la ligne dans la console avec un code couleur adapté :
 - Vert pour le niveau `Information`.
 - Jaune pour le niveau `Avertissement`.

- Rouge pour le niveau Erreur.

3. Sécurisation du traitement principal (Try / Catch)

- Consigner une entrée de journalisation de niveau Information annonçant le début de l'analyse sur le dossier cible.
- Encapsuler la lecture du dossier \$DossierCible dans un bloc try.
- Utiliser la commande de recherche de fichiers en forçant explicitement l'interruption immédiate en cas d'erreur (paramètre -ErrorAction Stop).
- Dans le bloc catch :
 - Récupérer le message d'erreur natif renvoyé par le système.
 - Consigner ce message via la fonction de journalisation au niveau Erreur.

4. Analyse et aiguillage des résultats

- Si la lecture du dossier réussit dans le bloc try :
 - Filtrer les fichiers dont la taille dépasse le seuil défini par \$SeuilTailleMo.
 - Si aucun fichier ne dépasse le seuil : consigner une mention de conformité au niveau Information.
 - Si des fichiers dépassent le seuil : consigner un message au niveau Avertissement indiquant le nombre total d'éléments détectés.
 - Si le commutateur \$ModeDetaillé est activé : consigner individuellement chaque fichier volumineux avec son nom complet et sa taille arrondie en Mo.

5. Bloc de clôture systématique (Finally)

- Déclarer un bloc finally associé à la structure de protection.
- Consigner un message de fin d'analyse au niveau Information, garantissant son exécution que le traitement ait réussi ou rencontré une exception.
- Afficher une notification finale en couleur cyan dans la console indiquant la fin du traitement.